DEEP LEARNING (DS306) · CODE ARCHITECTURE & TECHNICAL VIVA GUIDE

RecallAI: End-to-End Source Code & Deep Learning Walkthrough

Course Instructor: Dr. Kongara Srinivasa Rao | **Framework:** PyTorch · Transformers · Streamlit

Student 1: Golla Madhulika (23STUCHH010826)	**Student 2:** Reethu Reddy (23STUCHH011082)
Focus: Deep Learning Models, Math & Pipelines	**Files Covered:** 10 Modular Components (18 Tests Passing)

Part 1: The Deep Learning Architecture (Show These First)

1. src/text_embeddings.py — Dense Semantic Embeddings Engine

What to tell Sir: *"Sir, this is our mathematical foundation. Instead of using classical Bag-of-Words or TF-IDF which fail when synonyms are used, we utilize the Sentence-Transformers model all-MiniLM-L6-v2. It maps sentences into a continuous 384-dimensional dense semantic vector space where contextual similarity equals geometric proximity."*

```
def generate_embeddings(texts, model_name='all-MiniLM-L6-v2'):
    model = get_embedding_model(model_name)  # Cached SentenceTransformer instance
    embeddings = model.encode(texts, show_progress_bar=False,
                              convert_to_numpy=True, normalize_embeddings=True)
    return embeddings  # Shape: (N, 384) with unit L2 norm

def compute_cosine_similarity(vec_a, vec_b):
    # Cosine Similarity = (A . B) / (||A|| * ||B||)
    return np.dot(vec_a_norm, vec_b_norm.T)  # Pairwise matrix dot product
```

Key Technical Details to Highlight:

- **Transformer Architecture:** 6-layer Transformer, 12 attention heads, 22.7M parameters.
- **Mean Pooling:** Token embeddings produced by self-attention are averaged with attention masking.
- **Unit Normalization (L2):** Vectors are scaled so ||v|| = 1.0, meaning Cosine Similarity is simply the Dot Product.
- **Execution Time:** Runs entirely on CPU with PyTorch runtime, encoding a full page of notes in under 50ms.

2. src/topic_classifier.py — Zero-Shot NLI Classification

What to tell Sir: *"Sir, instead of training a custom supervised classifier which requires thousands of labeled examples and cannot generalize across college subjects, we formulate topic identification as Natural Language Inference (NLI) using typeform/distilbert-base-uncased-mnli (66M parameters). It evaluates entailment against 10 academic domains zero-shot."*

```
# Zero-Shot NLI Formulation
result = classifier(
    sample_text,
    candidate_labels=['Computer Science', 'Biology', 'Physics', 'Mathematics', ...],
    hypothesis_template='This educational text is about {}.'
)
# Model computes Softmax probability over Entailment vs Contradiction logits
```

Key Technical Details to Highlight:

- **NLI Mechanism:** Text is the *Premise*; candidate topics become the *Hypothesis*. The model calculates entailment logits.
- **DistilBERT Efficiency:** 40% smaller and 60% faster than BERT-base while retaining 97% of language understanding.
- **Secondary Centroid Fallback:** If NLI pipeline fails, line 148 calculates Temperature-Scaled Softmax (tau=0.15) over topic anchor embeddings.

3. src/summarizer.py — Centroid & Maximal Marginal Relevance (MMR)

What to tell Sir: *"Sir, generative models like ChatGPT hallucinate non-existent formulas or reword definitions inaccurately. In our summarizer, we guarantee 0% hallucination by selecting verbatim sentences mathematically using Document Centroid Vector and Maximal Marginal Relevance (MMR) with lambda = 0.75."*

```
# 1. Calculate Document Global Centroid Vector C (Mean of sentence embeddings)
doc_centroid = np.mean(sent_embs, axis=0, keepdims=True)
sims = compute_cosine_similarity(sent_embs, doc_centroid).flatten()

# 2. Maximal Marginal Relevance (MMR) Selection Loop
for _ in range(target_sentences):
    # Centrality = CosineSim(sentence, Centroid)
    # Redundancy = Max CosineSim(sentence, already_selected_sentences)
    mmr_score = lambda_param * centrality - (1 - lambda_param) * redundancy
    selected_indices.append(best_candidate_index)
```

Mathematical Formula:

MMR = argmax_{D_i in R \ S} [lambda * CosineSim(D_i, Centroid) - (1 - lambda) * max_{D_j in S} CosineSim(D_i, D_j)]

- **lambda = 0.75:** Balances high topic coverage (centrality) with strict non-redundancy (diversity).
- **Academic Integrity:** Every sentence is extracted verbatim, guaranteeing zero hallucinated facts.

4. src/question_generator.py — Vector-Space Distractor Synthesis

What to tell Sir: *"Sir, typical multiple choice question generators use random distractors that are trivially easy to eliminate. We use dense embedding cosine similarity to pick distractors from semantically proximate clusters (similarity between 0.15 and 0.94). This creates realistic, high-quality test questions that evaluate genuine conceptual understanding."*

```
# Vector Distractor Proximity Filter
target_emb = generate_embeddings([target_term])
cand_embs = generate_embeddings(candidate_terms)
sims = compute_cosine_similarity(target_emb, cand_embs)[0]

# Filter sweet spot: related enough to be plausible, distinct enough to not be duplicate
if 0.15 <= score <= 0.94 and term.lower() not in target_term.lower():
    valid_distractors.append((term, score))
```

Part 2: Walkthrough of All Other Supporting Modules

Module & File Path	Core Function & Code Highlights	What to Tell Sir
5. PDF Processor src/pdf_processor.py	extract_text_from_pdf() Uses PyMuPDF (pymupdf) to parse PDF byte streams, inspect encryption flags, detect scanned image PDFs, and extract page-by-page word counts.	*"Sir, PyMuPDF parses digital lecture PDFs directly in memory without saving temporary files, extracting raw text and page counts at C-speed."*
6. Text Cleaner src/text_cleaner.py	clean_text(), split_into_sentences() Strips page stamps (e.g. 'Page 1 of 5'), rejoins hyphenated breaks (trans-\nformer -> transformer), and protects abbreviations (Dr., e.g., vs., 3.14) during sentence splitting.	*"Sir, regex masks abbreviations so sentences aren't split prematurely, ensuring high embedding quality for the transformer."*
7. Flashcard Engine src/flashcards.py	generate_flashcards() Applies linguistic definition pattern heuristics (r'^(?P\w+)\s+is\s+(?P.+)') to pair academic concepts with their exact explanations.	*"Sir, this extracts term-definition pairs from lecture notes and creates flip-to-reveal study cards with mastery state tracking."*

8. Quiz Engine src/quiz_engine.py	create_quiz_session(), evaluate_quiz() Maintains active quiz state, calculates scores, and generates diagnostic feedback (e.g. tagging 'Backpropagation' for review if missed).	*"Sir, this module tracks quiz sessions and diagnoses weak concepts so the student gets targeted revision recommendations."*
9. UI Components src/ui_components.py	init_session_state(), inject_custom_css() Injects dark-mode academic CSS styling, renders the persistent sidebar with live session counters, and maintains multi-page session state.	*"Sir, this defines our modern matte charcoal academic UI inspired by modern Figma study apps, maintaining state across page reloads."*
10. Multi-Page Controller app.py & pages/	st.navigation([st.Page(...)]) Directs routing across 5 pages: (1) Dashboard, (2) Upload Notes, (3) Study Workspace, (4) Quiz, (5) Results & Progress.	*"Sir, app.py controls multi-page navigation using Streamlit's official navigation API, ensuring zero page-state desynchronization."*

Part 3: 5 Critical Viva Questions Sir Might Ask

Q1: Where exactly is the Deep Learning happening in your project?

Answer: Sir, primarily in 3 files: (1) src/text_embeddings.py where all-MiniLM-L6-v2 maps sentences into 384-dimensional dense vectors via self-attention and mean pooling; (2) src/topic_classifier.py where distilbert-base-uncased-mnli performs zero-shot cross-encoder inference; and (3) src/summarizer.py where MMR optimizes sentence cosine similarities against the document centroid.

Q2: Why did you choose Pretrained Zero-Shot models rather than training your own network?

Answer: Sir, university students study hundreds of diverse subjects (from Operating Systems to Organic Chemistry). Training a custom network would require millions of domain-labeled pairs and expensive GPU infrastructure. Pretrained transformers already possess generalized contextual semantic representations, allowing RecallAI to execute zero-shot inference across any syllabus instantly on standard CPU.

Q3: Why is your summarizer Extractive instead of Abstractive (like BART or T5)?

Answer: Sir, abstractive models generate new words and frequently hallucinate inaccurate formulas, false historical dates, or altered definitions. In technical subjects, student exam answers must match textbook rigor. Our MMR extractive summarizer mathematically selects verified verbatim sentences from the source notes, guaranteeing 0% hallucination.

Q4: What is the significance of the lambda parameter in your MMR summarizer?

Answer: Sir, lambda balances relevance versus redundancy. In our implementation (lambda = 0.75), 75% of the weight is given to choosing sentences closest to the document centroid C (high centrality), while 25% penalizes similarity to sentences already chosen (diversity). This prevents the summary from repeating the same concept in different words.

Q5: How do you handle speed and latency on a student's personal laptop?

Answer: Sir, both models are distilled: MiniLM has only 22.7M parameters and DistilBERT has 66M parameters (40% smaller and 60% faster than BERT-base). By running PyTorch inference on CPU with vector batching, the entire 5-step pipeline executes in under 2 seconds for a standard lecture note.